Inverse Reinforcement Learning for Resource Preference Modeling in Business Processes

Lukas Kirchdorfer
SAP Signavio, University of Mannheim
lukas.kirchdorfer@sap.com

Timotheus Kampik
SAP Signavio, Umeå University
timotheus.kampik@sap.com

Han van der Aa
University of Vienna
han.van.der.aa@univie.ac.at

Heiner Stuckenschmidt
University of Mannheim
heiner.stuckenschmidt@uni-mannheim.de

Abstract

Human resources are essential for successfully executing business processes. Unlike machines or software systems that operate according to predefined rules, humans make autonomous decisions during process execution, often reflecting individual preferences for particular types of work. Such preferences, for example regarding which work item to execute next, can significantly affect the trajectory, cost, and outcome of a process. Modeling them is therefore important not only for understanding resource behavior, but also for anticipating future choices to be made. Although event logs record the actions of resources over time, approaches that learn resource-specific preference models from such data remain scarce. We argue that Inverse Reinforcement Learning (IRL) provides a suitable foundation for this task: by interpreting the choices recorded in event logs as demonstrations, it enables the inference of reward functions that capture which options resources tend to prefer. Building on this idea, we develop an IRL-based approach to learning resource-specific preference models from event logs, focusing on case prioritization as a representative decision-making setting. Experiments on synthetic and real-world datasets demonstrate the predictive utility of our approach for anticipating resource choices, underscoring the potential of IRL for resource-centric process mining.

Keywords: Process mining, Inverse reinforcement learning, Resource-centric

1. Introduction

Organizations depend on business processes to deliver products and services to their customers. The execution of these processes involves a combination of human actors, software, and machines (Cabanillas, 2016; Dumas et al., 2013). Modern information systems play a central role in supporting these executions, as they record detailed data in the form of event logs, which provide the basis for process mining and data-driven analysis (van der Aalst, 2016). While research in process mining has traditionally focused on control-flow discovery and conformance checking, more recently, increasing attention is given to the *resource perspective* (Hoogmoed et al., 2025; Kirchdorfer et al., 2025; Klijn et al., 2024; Rubensson et al., 2024), which concerns the entities carrying out process activities. *Human* resources play a particularly influential role, as they not only execute work but may—depending on the type of process—also make autonomous decisions during process execution; increasingly, this may also apply to AI agents. They can, e.g., choose which case to prioritize, which activity to perform next, or with whom to collaborate (Rubensson et al., 2024). Such decisions often reflect resource preferences, i.e., tendencies of resources to favor particular types of work, collaboration partners, or execution strategies (Bidar et al., 2019). These preferences can shape process trajectories, costs, and outcomes, making them relevant for process analysis and improvement. Therefore, modeling resource preferences offers potential for understanding resource behavior, predicting future resource decisions, and improving the realism of process simulation models, among others.

Despite the growing interest in the resource perspective and the increasing number of methods to describe and characterize resources (cf. Rubensson et al. (2024) for an overview), learning resource preferences from event logs remains comparatively underexplored in process mining. A notable exception is the work

by Bidar et al. (2019), who discuss which types of preferences resources may exhibit in business processes and propose a supervised machine learning approach to predict, e.g., the next task a resource chooses from a worklist. While this constitutes an important step toward preference-aware process analysis, we argue that supervised learning is not necessarily the most natural modeling principle for this problem. In supervised learning, observed choices are typically treated as target labels to be predicted from a set of input features. However, preferences are about choosing one option over others: a resource selects a particular work item because it is preferred to the alternatives available at that moment. This perspective is closely aligned with imitation learning and, in particular, Inverse Reinforcement Learning (IRL) (Ng & Russell, 2000), where observed behavior is treated as demonstrations from which an underlying reward function is inferred.

Building on this perspective, we propose an IRL-based approach for resource preference learning in business processes. Our approach reconstructs resource-specific decision points from an event log and treats the observed choices as demonstrations of resource behavior. From these demonstrations, it learns a *reward function* that assigns weights, i.e., importance factors, to the observable aspects of the decision context. These weights can be used to compute a utility score for each of the alternatives available in the decision scenario. Therefore, the reward function serves as a resource-specific preference model that can be used to imitate and predict resource choices. Notably, our reward functions are linear in the feature space of the decision context, making them equally interpretable as, e.g., linear regression.

Our IRL-based approach can be instantiated for different resource decision-making situations, such as case prioritization or resource collaboration. In this paper, we focus on case prioritization as a representative setting, where resources choose which available case to execute next. Our evaluation with synthetic and real-life event logs demonstrates the predictive utility of our IRL-based approach, often outperforming rule-based heuristics and supervised machine learning models. Overall, our findings position IRL as a promising modeling paradigm for resource-centric process mining.

2. Background and Related Work

This section discusses related work on resource analysis in process mining and introduces IRL as the methodological basis of our approach.

Resource analysis in process mining. Learning and analyzing resource-related behavior from event logs has been studied in several sub-areas of process mining. *Resource analysis* explicitly targets behavioral and structural aspects of resources in business processes (Rubensson et al., 2024). Existing approaches characterize resources, for example, in terms of availability, productivity, workload, skill, experience, collaboration patterns, or task execution behavior (Hoogmoed et al., 2025; Pika et al., 2017). These works provide valuable insights into the resource perspective, but they do not focus on resource-specific preference models.

The notion of *preference* has been considered and recognized as an underlying factor in several works on resource allocation and business process automation (Huang et al., 2012; Sindhgatta et al., 2016; Vanderfeesten & Grefen, 2015; Zhao et al., 2016). However, these works mainly treat preference as an allocation criterion, resource characteristic, or behavioral measure, rather than learning resource-specific preference models from data. Most closely related to our work is Bidar et al. (2019), who provide a conceptual account of preferences in business process automation and distinguish between resource-task, resource-resource, and task-resource preferences. They further show how certain forms of preferences can be derived from event logs using supervised machine learning, specifically Support Vector Machines (SVM), for example, by predicting the next task a resource selects from a worklist.

Our work builds on this line of research, but adopts a different modeling principle. Rather than treating preference learning solely as a supervised prediction problem, we model observed resource choices as demonstrations of behavior and learn reward functions that assign utility to the alternatives available in a decision situation. In this way, the learned reward functions serve as resource-specific preference models that can be used to imitate and predict resource choices. This also distinguishes our work from related areas in process mining such as *organizational mining*, which focuses on organizational structures and social networks (Song & van der Aalst, 2008), *agent system mining*, which aims to discover agent systems from process data (Tour et al., 2021), *user behavior mining*, which focuses on user–software interactions (Rehse et al., 2024), and *decision mining*, which derives rules for control-flow branching (Rozinat & van der Aalst, 2006). While these areas are related, our focus is on learning resource-specific preference models for decisions in which a resource selects one option from a set of available alternatives.

Inverse Reinforcement Learning. Reinforcement Learning (RL) models sequential decision-making,

where an agent interacts with an environment by taking actions in different states. Given a predefined reward function, the agent learns a policy that specifies which action to take in order to maximize cumulative reward (Sutton & Barto, 2018). The reward function is therefore central to the problem formulation, as it defines which behavior is desirable.

However, in many real-world settings—particularly those involving human decision-making, such as business process execution—the reward function is not known a priori and often very difficult to define. For example, in a loan application process, it is unclear whether a clerk prioritizes customers to minimize waiting times, to focus on high-value loans, or simply to reduce personal workload. Such objectives may vary across individuals, change over time, and even conflict with organizational policies, making it virtually impossible to handcraft an accurate reward function.

Inverse Reinforcement Learning (IRL) (Ng & Russell, 2000) bypasses the need to manually specify a reward function and instead infers it from observations, such as contained in event logs used in process mining. The goal of IRL is therefore not merely to reproduce observed actions, but to infer the preferences or principles that make these actions likely in a given decision context. As emphasized by Ng and Russell, "the reward function, rather than the policy, is the most succinct, robust, and transferable definition of the task" (Ng & Russell, 2000). In our setting, the learned reward function quantifies how desirable different alternatives are for a resource and thus provides a natural representation of its preferences.

IRL has been applied in various domains such as robotics and autonomous driving (see Adams et al. (2022) for a review). However, despite its suitability for learning from observed choices, its potential for process mining has not yet been explored.

3. Motivation

To illustrate the motivation behind our work, we consider a simplified loan application process. Applications from prospective customers arrive continuously and are processed by employees (Alice, Bob, and Carol) in the credit department. At any point in time, multiple applications may be waiting to be processed, while employees can choose which case to handle next. This decision problem, known as *case prioritization*, is common across many business processes and has been studied frequently in literature (Lashkevich et al., 2024; Pika et al., 2017; Rubensson et al., 2024; Senderovich et al., 2014). Case prioritization is important because the order in which cases are handled affects waiting times, queue dynamics, and ultimately the cycle times of cases.

Table 1: Excerpt of motivating event log. The *queue* represents the choice of temporally ordered case IDs available to the resource at the respective event. The final row illustrates a future decision point for which the selected case is unknown and has to be predicted.

Case	Activity	Start	End	Resource	Amount	Queue
1	Check	09:12	09:55	Alice	50k	[1,2,3,4,5,6]
5	Check	09:15	10:10	Bob	80k	[2,3,4,5,6]
3	Check	09:18	10:15	Carol	2k	[2,3,4,6]
2	Check	10:20	11:30	Alice	10k	[2,4,6]
6	Check	10:23	11:20	Bob	70k	[4,6]
4	Check	10:26	11:45	Carol	3k	[4]
				...		
?	Check	12:05	?	Bob	?	[7 (5k); 8 (90k)]

Table 1 presents an excerpt of an event log from the corresponding process. For each event, the queue[1] indicates the cases that were available to the resource at the time of selection. At 09:12, Alice selects Case 1 from a queue of six pending applications, i.e., the oldest available case. This suggests a preference consistent with a first-in–first-out (FIFO) strategy. By contrast, Bob selects Case 5 and later Case 6, both of which have comparatively high loan amounts. Carol, in turn, selects Cases 3 and 4, which have comparatively low loan amounts.

While this small example is only illustrative, it shows how repeated choices among available alternatives may reveal resource-specific preferences. Being aware of these preferences is an important instrument for process analysis and decision support: it allows organizations to identify systematic selection patterns, compare them with intended prioritization rules, and derive policies that better align resource choices with organizational goals, such as prioritizing high-value customers to ensure fast processing and a positive customer experience.

Beyond such retrospective analysis, preference models are particularly useful to anticipate future resource choices. The final row illustrates the corresponding prediction problem. Given Bob's previous choices and a new queue containing Cases 7 and 8, which case is Bob likely to select next? A FIFO-based model would favor Case 7, as it is the oldest case in the queue. A resource-specific preference model, however, may assign a higher probability to Case 8 if it has learned that Bob tends to prioritize high-value loans. Thus, preferences can support predictive process monitoring tasks, such as remaining time prediction

[1]Note that the queue is usually not part of an event log, but can be derived based on the time information.

or the early detection of cases that are likely to be delayed or risk violating service-level agreements. The same argument applies to simulation, where realistic representations of resource choices are essential for generating plausible process executions (Dumas, 2021).

The above example highlights two requirements for modeling resource choices: *(i)* preferences are resource-specific: different employees may choose differently even when facing similar decision situations; *(ii)* preferences are choice-set dependent: a case is not selected in isolation, but from a set of alternatives available at a particular point in time. Hence, predicting future resource decisions requires models that account for both the resource and the decision context.

IRL offers a principled way to learn such preference models from event logs. By treating observed resource choices as demonstrations, it infers reward functions that assign utility to available alternatives. These reward functions can then be used to estimate which option a resource is likely to select in a future decision situation. In addition, their linear form allows the learned preference weights to be inspected to analyze the factors that drive the preference.

4. Approach

This section introduces our IRL-based approach for learning resource preferences from event logs. Each resource involved in the process is modeled as an agent, for which a distinct reward function is learned. The following outlines the approach's input, main steps, and output, as visualized in Figure 1.

Input. Our approach takes as input an event log L, defined as a multiset of events. Each event $e \in L$ is represented as a tuple $e = (c_{id}, act, t_s, t_e, r, d_1, \ldots, d_p)$, where c_{id} is the case identifier, act the activity label, t_s the start timestamp, t_e the completion timestamp, and r the resource that executed the event. We use $\mathcal{R}_L = \{r \mid (c_{id}, act, t_s, t_e, r, d_1, \ldots, d_p) \in L\}$ to specify the set of resources observed in the event log L. The additional attributes $d_1, \ldots, d_p$ encode further information such as the requested loan amount.

Approach steps. The overarching goal is to learn resource preferences from an event log by modeling resources as agents. To achieve this, our approach consists of two main steps. Given a log L, Step 1 extracts trajectories for each agent individually, which are temporally ordered sequences of state–action pairs, capturing all decision points of an agent recorded in the process. Step 2 applies IRL to these trajectories in order to infer an agent's reward function.

Output. As output, our approach yields a set of

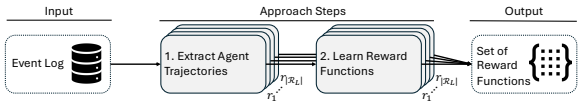


Figure 1: Overview of the approach. The two steps are executed per agent $r \in \mathcal{R}_L$.

reward functions, $\mathrm{R} = \{R_r \mid r \in \mathcal{R}_L\}$, where each $R_r : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ denotes the reward function of resource r, constituting its preference model. Here, $\mathcal{S}$ and $\mathcal{A}$ represent the agent's state and action spaces, respectively (whose construction we describe further below). Intuitively, $R_r(s, a)$ quantifies the utility that resource r associates with performing action $a \in \mathcal{A}$ in state $s \in \mathcal{S}$. For instance, in the loan application scenario from Section 3, the state comprises the available applications represented by some attributes (e.g., loan value and waiting time), while the action corresponds to the selection of a specific application. A resource favoring high-value loans would thus receive higher rewards for choosing such applications.

Sections 4.1 and 4.2 describe the two main steps of our approach in more detail.

4.1. Step 1: Extracting Agent Trajectories from the Event Log

The first step of our approach transforms the event log L into a set of trajectories, one for each agent $r \in \mathcal{R}_L$. This requires interpreting the event log as a sequence of decision-making scenarios faced by the agents. From this perspective, we derive the states that capture the decision context at each point in time and the corresponding actions taken by the agents, which together form the trajectories.

Decision-making perspective. To formulate resource preference learning within the framework of IRL, we interpret the event log as the outcome of sequential decision-making by multiple agents. Since this decision-making perspective is not explicitly recorded in the log, it must be reconstructed by defining the *state* $s \in \mathcal{S}$, representing the decision context available to an agent at a given point in time, and the *action* $a \in \mathcal{A}$, representing the choice the agent made in that context. A prerequisite for this reconstruction is to specify the type of decision to be modeled, since humans may face different decision-making situations within the same process. For example, deciding which activity to perform next in a case, to whom a case should be handed over, or which case to prioritize. While our approach is general and applicable to all of these perspectives—simply by changing what is represented

in the state and action spaces—we introduce it using the case prioritization problem as our running example.

Constructing states from event data. For constructing the different states from L, we sort the events in temporal order based on their start timestamp and reconstruct, for each decision point, the set of alternatives available to the resource at the moment of case selection, i.e., at start time t_s of the event. The precise notion of availability depends on the granularity of the modeled decision problem. We adopt case-level availability, considering a case available if it has already arrived and no resource is currently processing an activity of that case. This constitutes a simplifying assumption: processes with concurrently enabled activities of the same case would require reconstructing alternatives at the work-item level, which we leave for future work.

We define the state $s \in \mathcal{S}$ as the feature-based representation of this queue. The feature representation determines which preference factors can be captured by the learned reward function and should therefore be chosen with respect to the decision problem under study. In general, features should (i) be observable at the time the resource makes the decision, (ii) plausibly influence the relative desirability of the available alternatives, and (iii) discriminate between alternatives within a choice set. For case prioritization, relevant features may include case or work-item characteristics (e.g., activity or customer type), temporal and urgency-related information (e.g., waiting time or due date), and decision-history information (e.g., the resource that previously handled a case). Features available only after the decision should be excluded to prevent information leakage. Consequently, there is no universally applicable feature set. Its composition depends on the process, the modeled decision, and the information recorded in the event log.

Extracting actions from event data. In the context of the case prioritization problem, an action corresponds to selecting a specific case from the queue. More formally, the action a taken by agent r at time t_s thus corresponds to the case identifier c_{id} of the event that r starts at time t_s. By pairing the reconstructed state s with the observed action a, we obtain a state–action pair (s, a).

Note that if a different decision-making problem were modeled—such as selecting the next resource for a handover—only the definitions of state and action would change: the action would be the choice of a resource from the set of available ones, while the state could capture factors such as current workloads and the activity to be performed.

Agent trajectories. By extracting the state s and the action a for each event e, executed by agent $r \in \mathcal{R}_L$, from our temporally ordered event log L, we obtain a sequence of N_r state–action pairs:

$$\tau_r = \langle (s_1, a_1), (s_2, a_2), \ldots, (s_{N_r}, a_{N_r}) \rangle. \quad (1)$$

Repeating this for all agents yields the set of agent trajectories $\mathcal{T} = \{\tau_r \mid r \in \mathcal{R}_L\}$, which serves as the demonstration data for the subsequent IRL step.

Illustrative example. For the motivating scenario in Table 1, suppose that at time t_s = 10:23 two loan applications are waiting in the queue: Case 4 (loan amount 3,000; waiting 4 hours) and Case 6 (loan amount 70,000; waiting 2 hours). Since Bob selects Case 6, the reconstructed state s encodes the features of both cases, while the action a corresponds to choosing Case 6. By doing this for every observed decision in the log, we obtain a trajectory that captures Bob's decision-making as sequences of state–action pairs.

4.2. Step 2: Learning the Reward Functions

Having extracted the set of agent trajectories $\mathcal{T}$, containing sequences of state–action pairs, the next step is to infer a reward function for each agent $r \in \mathcal{R}_L$ that constitutes its preference model. IRL provides a family of methods for this purpose, each differing in how it resolves the inherent ambiguity of reward recovery. Therefore, we first review existing IRL methods and motivate our choice of the most suitable one. We then present how this method can be adapted to the process mining setting, define the resulting reward function and policy, and explain how the reward weights are learned.

A brief discussion of IRL methods. Since recovering reward functions from demonstrations is an ill-posed problem—for any given policy there exist infinitely many reward functions that could rationalize it (Ng & Russell, 2000)—different IRL approaches impose distinct assumptions to select a solution. Prominent examples include the maximum margin principle (Abbeel & Ng, 2004), Bayesian formulations (Ramachandran & Amir, 2007), adversarial methods such as Generative Adversarial Imitation Learning (GAIL) (Ho & Ermon, 2016), and the maximum entropy principle (Ziebart et al., 2008). While each approach has certain advantages, they come at the cost of either reduced robustness to noise (max-margin), computational overhead (Bayesian), or loss of interpretability (GAIL).

We adopt the principle of *Maximum Entropy IRL* (MaxEnt IRL) (Ziebart et al., 2008). MaxEnt IRL selects the reward function that makes the observed behavior most likely, while otherwise remaining maximally unbiased about unobserved choices. It

has several advantages for our setting: (i) it yields an interpretable linear reward function over the feature space, (ii) it is more efficient than Bayesian alternatives, and (iii) it naturally accommodates suboptimal or noisy demonstrations better than max-margin methods (Adams et al., 2022).

Simplifying from dynamic MDPs to static decisions. Classical IRL, including MaxEnt, is formulated in the context of Markov Decision Processes (MDPs), where agents optimize cumulative long-term rewards over state transitions. In business processes, however, we assume that many resource decisions are inherently myopic (i.e., short-sighted): for example, in case prioritization, an agent selects one case from the queue based on its current features, without explicit consideration of future states. To capture this property, we adapt MaxEnt IRL to a simplified setting of independent decision points, as explained in detail next. This adaptation makes the method more efficient and aligned with the structure of event log data, while retaining the central benefit of MaxEnt IRL: providing reward functions that are both probabilistically grounded and interpretable in terms of feature weights. However, our approach can be extended to the MDP setting; we leave this for future work.

Reward and policy. Formally, at each decision point, we observe a state $s \in \mathcal{S}$, describing the available cases in the queue, and a set of actions $\mathcal{A}_s$, where each action $a \in \mathcal{A}_s$ corresponds to selecting one of these cases. Each action is described by a feature vector $\phi(s,a) \in \mathbb{R}^k$ consisting of k distinct features. We assume that each agent $r \in \mathcal{R}_L$ follows a linear reward function

$$R_r(s,a) = \theta_r^\top \phi(s,a), \quad \theta_r \in \mathbb{R}^k, \tag{2}$$

where the learnable parameter vector θ_r encodes the relative importance of the features for agent r. Following the maximum entropy principle, the probability of agent r selecting action a in state s is modeled by the Boltzmann distribution:

$$\pi_{\theta_r}(a \mid s) = \frac{\exp(\theta_r^\top \phi(s,a))}{Z_s(\theta_r)}. \tag{3}$$

Here, $Z_s(\theta_r)$ denotes the normalization term, also known as the partition function:

$$Z_s(\theta_r) = \sum_{b \in \mathcal{A}_s} \exp(\theta_r^\top \phi(s,b)). \tag{4}$$

It sums the exponentiated rewards of all actions available in state s, ensuring that the probabilities assigned by π_{θ_r} over all actions in $\mathcal{A}_s$ sum to one.

Intuitively, the reward function assigns a numerical score to each possible action in the queue based on the weighted combination of its features. The policy π_{θ_r} then converts these scores into choice probabilities: actions with higher rewards are more likely to be selected, while lower-scoring actions still retain non-zero probability. This stochastic choice model reflects that human decisions are not perfectly deterministic, while still favoring actions that receive higher rewards under θ_r.

Learning the reward weights. Given $\tau_r = \langle (s_1,a_1),(s_2,a_2),\ldots,(s_{N_r},a_{N_r}) \rangle$ as the demonstrations of agent r, we estimate the parameters θ_r of its reward function by maximizing the likelihood that the model reproduces the observed decisions. Intuitively, this means adjusting the model such that the actions actually taken by the resource appear most probable under the learned model, yielding the log-likelihood objective:

$$\mathcal{L}(\theta_r) = \sum_{i=1}^{N_r} \left(\theta_r^\top \phi(s_i,a_i) - \log Z_{s_i}(\theta_r) \right). \tag{5}$$

To find the parameters that maximize this objective, we compute its gradient with respect to θ_r, which indicates how the parameters should be adjusted to increase the likelihood of the observed decisions. The resulting expression takes the familiar form from MaxEnt IRL, i.e., $\nabla \mathcal{L}(\theta_r)$ equates

$$\underbrace{\sum_{i=1}^{N_r} \phi(s_i,a_i)}_{\text{empirical feature sum}} - \underbrace{\sum_{i=1}^{N_r} \sum_{a \in \mathcal{A}_{s_i}} \pi_{\theta_r}(a \mid s_i)\, \phi(s_i,a)}_{\text{model-expected feature sum}}. \tag{6}$$

In practice, the parameters θ_r are updated iteratively over multiple epochs, each time reducing the discrepancy between the empirical and model-expected feature sums. Through this process, the model gradually aligns its predicted behavior with the observed trajectories of the resource. Once convergence is reached, the learned parameters define the reward function R_r that maximizes the likelihood of the observed decisions. Repeating this learning procedure for each agent results in a set of reward functions $\mathrm{R} = \{R_r \mid r \in \mathcal{R}_L\}$ that constitute the preference models of all resources in the process.

Prediction. Once the reward functions have been learned, they can be used to predict resource decisions in unseen situations. Given a new state s, we compute $R_r(s,a)$ for all available actions $a \in \mathcal{A}_s$ and use the resulting policy $\pi_{\theta_r}(a \mid s)$ to obtain a probability distribution over the available actions. Actions could be either sampled from this distribution or, as we do in our evaluation, ranked by their predicted probability.

5. Experiments and Results

This section describes the experimental evaluation of our approach to learning resource preference models in the form of reward functions from event logs. Section 5.1 describes the experimental setup, followed by the results in Section 5.2. Our approach implementation (based on Python 3), our synthetic event logs, and supplementary results are available in our public repository.[2]

5.1. Experimental Setup

We evaluate our approach on both synthetic and real-world event logs, using case prioritization as the underlying decision-making task. The primary goal of the evaluation is to assess the predictive utility of the learned preference models, i.e., their ability to anticipate which case a resource selects from a queue of available cases. The synthetic logs allow us to evaluate this under different controlled conditions, where the resource choices are generated according to known prioritization strategies.

Event logs. Our evaluation is based on a diverse set of 10 event logs, eight of which are synthetically created.

Synthetic logs. For the eight synthetic logs, we developed an event log generator that produces variants of the loan application process introduced in Section 3, each consisting of 1000 cases. As each case comprises a single activity, this results in 1000 decision points per log. The size of the choice set (i.e., the number of cases available for selection at a given time) varies over time and averages 32 cases. The process involves three resources—Alice, Bob, and Carol—who handle incoming loan applications. Each case, i.e., each application, is described by several features: a loan value, the customer status (*platinum*, *gold*, or *silver*), the region of the customer (*EMEA*, *US*, or *Africa*), the expected processing time (PT), the elapsed time since the application was submitted (TSA), and the time until the application is due (TUDD). To evaluate predictive performance under different preference patterns, we generated eight variants of this process, each based on a specific case prioritization strategy. These strategies comprise four standard scheduling policies, as outlined by Silver et al. (1998), together with four process-specific ones:

- **EDD (Earliest Due Date)**: Resources always handle the case with the earliest due date first.
- **FCFS (First Come, First Served)**: Resources work on the case that has been waiting in the queue the longest.
- **SPT (Shortest Processing Time)**: Resources select the case with the shortest expected processing time.
- **Random**: Resources choose cases at random.
- **Value**: Resources give priority to high-value applications.
- **Platinum**: Resources prioritize applications from platinum-level customers.
- **Mix-1**: Each resource follows a different policy—Carol prioritizes high-value customers, Bob exclusively serves US customers, and Alice prioritizes platinum-level customers.
- **Mix-2**: Each resource follows a different policy—Carol follows the SPT policy, Alice the FCFS policy, and Bob prioritizes high-value customers.

Real-life logs. To also test the performance of our approach in real scenarios, we follow Bidar et al. (2019) and use two real-life logs that contain resource and task life cycle information required for extracting the queues at each decision point. For both logs, we select a compact set of interpretable features that are available at decision time and plausibly characterize differences between the cases from which a resource chooses. First, the BPIC2012W[3] log represents a loan application process of a Dutch financial institution. Here, we use *activity label* to capture work-item type, *loan amount* as a central case attribute, and *time since arrival* to capture how long the work item has been waiting for processing. Second, the BPIC2013 (Incidents)[4] log stems from a Volvo IT incident management system. Here, we use *product label* and *incident impact* to characterize the incident, together with *time since arrival*. In addition, we include the *previous resource* working on the case to capture its execution history. This may be relevant in the incident-management setting, following previous analyses of the log that identified frequent reassignment and "ping-pong" behavior between support teams. In contrast, we did not include such a history feature for BPIC2012W, for which we focus on characteristics of the current work item and its waiting time. Thus, the feature sets constitute process-specific modeling choices rather than a universal feature representation. For BPIC2012W / BPIC2013, we consider 9,270 / 11,497 decision points with an avg. queue size of 14 / 329 cases.

Benchmark approaches. We compare our IRL approach against a diverse set of rule-based and tabular ML benchmarks to predict the next case in a queue, applied individually to each agent:

- **Random**: Selects cases uniformly at random.
- **FIFO**: Aligns with the common FCFS rule, selecting

[2] https://github.com/lukaskirchdorfer/IRL4PM

[3] https://data.4tu.nl/articles/dataset/BPI_Challenge_2012/12689204/1

[4] https://data.4tu.nl/articles/dataset/BPI_Challenge_2013_incidents/12693914

Table 2: Average top-k accuracies. Best results per dataset are highlighted in bold, the second-best underlined.

	Random		FIFO		DT		SVM		LR		XGB		NN		IRL (Ours)	
	$k=1$	$k=3$	$k=1$	$k=3$	$k=1$	$k=3$	$k=1$	$k=3$	$k=1$	$k=3$	$k=1$	$k=3$	$k=1$	$k=3$	$k=1$	$k=3$
EDD	0.06	0.23	0.20	0.54	0.16	0.31	0.30	0.64	0.33	0.65	0.23	0.51	0.18	0.45	**0.69**	**0.98**
FCFS	0.22	0.55	**1.00**	**1.00**	0.16	0.44	0.33	0.71	0.29	0.69	0.27	0.66	0.24	0.62	0.51	0.94
SPT	0.06	0.11	0.01	0.02	0.16	0.47	0.57	0.86	0.63	0.91	0.42	0.75	0.22	0.46	**0.68**	**0.94**
Rand.	0.08	0.24	0.09	0.25	**0.11**	0.20	0.06	**0.27**	0.05	0.22	0.07	0.22	0.08	0.23	0.09	0.26
Value	0.03	0.10	0.02	0.04	0.09	0.20	0.56	**0.85**	0.55	0.84	0.44	0.67	0.21	0.42	**0.59**	0.84
Plat.	0.04	0.13	0.04	0.11	0.16	0.30	0.26	0.38	**0.31**	**0.44**	0.26	0.40	0.19	0.39	0.29	0.43
Mix-1	0.03	0.06	0.22	0.23	0.28	0.36	0.35	0.52	0.36	**0.58**	0.25	0.46	0.18	0.38	**0.41**	0.54
Mix-2	0.09	0.18	0.34	0.35	0.48	0.78	0.67	**0.98**	0.71	**0.98**	0.55	0.78	0.43	0.69	**0.86**	**0.98**
BPI12W	0.10	0.30	0.22	0.35	0.31	0.46	0.29	0.57	0.23	0.49	**0.36**	**0.62**	0.27	0.51	0.28	0.53
BPI13	0.01	0.03	0.01	0.01	**0.36**	**0.36**	0.18	0.29	0.22	0.34	0.15	0.21	0.20	0.31	0.16	0.28
Average	0.07	0.19	0.22	0.29	0.23	0.39	0.36	0.61	0.37	0.61	0.30	0.53	0.22	0.45	**0.46**	**0.67**

cases that arrived first.

- **Decision Tree (DT)**: Interpretable model to learn case prioritization; used in recent simulation-based prioritization work (Lashkevich et al., 2024).
- **Support Vector Machine (SVM)**: Margin-based classifier; included as a supervised learning baseline in line with Bidar et al. (2019)
- **Logistic Regression (LR)**: Simple and interpretable linear model.
- **XGBoost (XGB)**: Gradient-boosted trees.
- **Neural Network (NN)**: Feed-forward network.

All learning-based approaches are trained and evaluated per resource on the same reconstructed decision points.

Data split. For each event log, we perform a temporal train/validation/test split with the first 64% of cases being in the training set, the following 16% being in the validation set, and the last 20% of cases in the test set.

Hyperparameters. We tune all ML models via grid search and report test results for the configuration with the best validation performance. Our approach is trained for 100 epochs; the only tuned hyperparameter is the learning rate $\in \{0.001, 0.005, 0.01, 0.05, 0.1\}$. For DT, we vary max depth $\in \{5, 10, 20, \infty\}$ (with ∞ denoting no limit). For SVM, we tune $C \in \{0.1, 1, 10\}$ (controlling the regularization) and the kernel (linear or RBF). For XGB, we tune the learning rate $\in \{0.01, 0.05, 0.1\}$. For NN, we tune the architecture: number of layers and neurons per layer $\in \{50/25, 100/50, 100/50/25\}$.

Metrics. To evaluate and compare predictive performance in anticipating resource choices, we use top-k accuracy with $k \in \{1, 3\}$. Top-1 accuracy measures how often the model's most likely prediction exactly matches the actual case selected by the resource. Top-3 accuracy relaxes this requirement, assessing whether the correct case is included within the model's top three predictions, respectively. These metrics capture both the precision of the prediction and the usefulness of the ranked recommendations, which is particularly relevant in practical scenarios where decision support systems may suggest a shortlist of cases rather than a single choice.

5.2. Results

Overall results. Table 2 reports the top-1 and top-3 accuracies across the 10 event logs and 8 evaluated approaches. Overall, our IRL-based approach achieves the best or second-best performance in 8 out of 10 logs. Averaged across all logs, it reaches top-1 and top-3 accuracies of 46% and 67%, respectively, substantially outperforming the second-best approach, LR, which achieves 37% and 61%. Two notable exceptions among the synthetic logs where IRL does not achieve the top performance are *FCFS*, where FIFO achieves the expected 100% accuracy, and *Random*, where no systematic preference signal is present and all approaches perform similarly poorly.

The synthetic results show that our approach performs well across different types of prioritization strategies. It achieves strong results for strategies based on numerical features, such as *EDD*, where case selection depends on the time until the due date, and *SPT*, where the expected processing time determines the choice. At the same time, it also performs strongly in settings governed by categorical preferences, such as *Mix-1* or *Platinum*. In this setting, also tree-based models such as DT and XGB perform competitively, as they can naturally partition the feature space according to categorical conditions. Nevertheless, the IRL-based model and LR achieve the highest top-1 and top-3 accuracies. This indicates that modeling resource choices through a reward function over the available alternatives is effective across different types of preference patterns. The task is also non-trivial,



Figure 2: Learned reward weights of Carol for six synthetic logs. Green squares denote the ground-truth dominant feature that guides the agent's prioritization.

as queues frequently contain many candidate cases, making accurate top-1 predictions challenging.

The results on the real-life logs provide a more nuanced picture. On both BPI12W and BPI13, some of the supervised learning approaches outperform IRL, with XGB and DT achieving the best performance, respectively. Nevertheless, IRL remains competitive and clearly outperforms simple scheduling heuristics such as FIFO on both logs. Compared with the controlled synthetic settings, real-world resource decisions may depend on unobserved factors and exhibit greater behavioral noise and randomness, which may limit the extent to which they can be captured by the learned reward function. At the same time, the results show that data-driven models can capture resource selection behavior substantially better than fixed scheduling rules. Since FIFO remains a common default scheduling policy in business process simulation models (Dumas, 2021; Kirchdorfer et al., 2025), learning resource-specific selection behavior from event data may provide a more realistic basis for representing case prioritization.

Supplementary analysis of learned reward weights. The linear reward functions also allow us to inspect the learned feature weights. For the synthetic logs, where the data-generating prioritization strategies are known, this allows us to verify whether the learned models capture the intended preference signal. For clarity, we report the learned weights in Figure 2 only for the six logs in which all three agents follow the same prioritization strategy and show the reward function learned for Carol as a representative example. The corresponding weights for Bob and Alice are provided in the accompanying repository.

The learned weights generally reflect the prioritization strategies used to generate the synthetic logs. In the *FCFS* log, the largest positive weight is assigned to TSA, indicating that cases with longer waiting times receive higher utility. In the *EDD* log, the model assigns a large negative weight to TUDD, which is consistent with prioritizing cases with earlier due dates. Similarly, the dominant features in the *SPT* and *Value* logs correspond to processing time and loan value, respectively. As expected, the *Random* log does not exhibit a clear weighting pattern, reflecting the absence of a systematic prioritization rule.

For the *Platinum* log, the model assigns a positive weight to the platinum feature and much smaller weights to gold and silver, which is consistent with the intended preference for platinum customers.

6. Conclusion

This paper presented an approach based on Inverse Reinforcement Learning (IRL) that enables the extraction of resource preference models from business process event logs. By reconstructing resource-specific decision points and learning reward functions from observed choices, our approach models how resources select among the alternatives available in a decision situation. The learned reward functions serve as resource-specific preference models that can be used to predict future resource choices. Experiments on synthetic and real-life event logs demonstrate the predictive utility of the approach, showing that it can accurately anticipate case prioritization decisions across different types of preference patterns and real-world settings, often outperforming the considered supervised machine learning baselines.

As the first application of IRL in process mining, this work naturally has several limitations and opens different directions for future research. First, our evaluation—while indicating the strong potential of IRL—is still limited in scope. For instance, we focus on case prioritization as one relevant and common resource decision problem, but future work should evaluate the approach for other types of resource decisions, such as resource handover. Moreover, the synthetic logs are useful for controlled evaluation, but necessarily simplify real-world behavior. More complex settings, for example, involving competing or changing preference patterns, could provide a more fine-grained assessment. Second, we currently assume myopic decision-making and use a simplified IRL formulation. Future work could investigate settings in which resources act more strategically and current choices affect future decision opportunities. Third, our approach relies on the assumption that the event log captures all attributes relevant to decision-making;

unobserved factors may therefore be reflected only indirectly through proxy correlations. Fourth, our approach assumes preferences to remain static. Future work could study dynamically changing preferences, for instance when resources adapt their behavior over time or under changing workloads. Further promising directions include incorporating learned preference models into business process simulation and evaluating their impact on downstream predictive monitoring tasks.

Finally, while a resource-centered viewpoint offers clear advantages for examining processes, it can also raise privacy concerns. We therefore advocate applying our work to process-level decision-making rather than to scrutinizing the behavior of individual employees. This can be achieved, e.g., by anonymizing reported results or abstracting them to process-level statements, such as indicating that not all clerks follow an organization's FIFO policy and rather favor high-value customers.

References

Abbeel, P., & Ng, A. Y. (2004). Apprenticeship learning via inverse reinforcement learning. *ICML*, 1.

Adams, S. C., Cody, T., & Beling, P. A. (2022). A survey of inverse reinforcement learning. *Artif. Intell. Rev.*, *55*(6), 4307–4346.

Bidar, R., ter Hofstede, A. H. M., Sindhgatta, R., & Ouyang, C. (2019). Preference-based resource and task allocation in business process automation. *OTM*, *11877*, 404–421.

Cabanillas, C. (2016). Process-and resource-aware information systems. *EDOC*, 1–10.

Dumas, M. (2021). Constructing digital twins for accurate and reliable what-if business process analysis. *BPM Workshops*, *2938*, 23–27.

Dumas, M., Rosa, M. L., Mendling, J., & Reijers, H. A. (2013). *Fundamentals of business process management*. Springer.

Ho, J., & Ermon, S. (2016). Generative adversarial imitation learning. *NeurIPS*, 4572–4580.

Hoogmoed, A., Djurica, D., Vidgof, M., et al. (2025). Enhancing process mining with visual resource analytics. *Software and Systems Modeling*.

Huang, Z., Lu, X., & Duan, H. (2012). Resource behavior measure and application in business process management. *ESWA*, *39*(7).

Kirchdorfer, L., Blümel, R., Kampik, T., van der Aa, H., & Stuckenschmidt, H. (2025). Discovering multi-agent systems for resource-centric business process simulation. *Process Sc.*, *2*, 4.

Klijn, E. L., Tentina, I., Fahland, D., & Mannhardt, F. (2024). Decomposing process performance based on actor behavior. *ICPM*, 129–136.

Lashkevich, K., Milani, F., Chapela-Campa, D., Suvorau, I., & Dumas, M. (2024). Unveiling the causes of waiting time in business processes from event logs. *Inf. Syst.*, *126*, 102434.

Ng, A. Y., & Russell, S. (2000). Algorithms for inverse reinforcement learning. *ICML*.

Pika, A., Leyer, M., Wynn, M. T., Fidge, C. J., ter Hofstede, A. H. M., & van der Aalst, W. M. P. (2017). Mining resource profiles from event logs. *TMIS*, *8*(1), 1:1–1:30.

Ramachandran, D., & Amir, E. (2007). Bayesian inverse reinforcement learning. *IJCAI*, 2586–2591.

Rehse, J., Abb, L., Berg, G., Bormann, C., Kampik, T., & Warmuth, C. (2024). User behavior mining. *Bus. Inf. Syst. Eng.*, *66*(6), 799–816.

Rozinat, A., & van der Aalst, W. (2006). *Decision mining in business processes*. Technische Universiteit Eindhoven.

Rubensson, C., Pufahl, L., & Mendling, J. (2024). A conceptual framework for resource analysis in process mining. *EDOC Workshops*.

Senderovich, A., Weidlich, M., Gal, A., & Mandelbaum, A. (2014). Mining resource scheduling protocols. *BPM*, 200–216.

Silver, E. A., Pyke, D. F., & Peterson, R. (1998). *Inventory management and production planning and scheduling* (3rd). John Wiley & Sons.

Sindhgatta, R., Ghose, A. K., & Dam, H. K. (2016). Context-aware analysis of past process executions to aid resource allocation decisions. *CAiSE*, 575–589.

Song, M., & van der Aalst, W. M. P. (2008). Towards comprehensive support for organizational mining. *Decis. Support Syst.*, *46*(1), 300–317.

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction*. A Bradford Book.

Tour, A., Polyvyanyy, A., & Kalenkova, A. A. (2021). Agent system mining: Vision, benefits, and challenges. *IEEE Access*, *9*, 99480–99494.

van der Aalst, W. M. P. (2016). *Process mining - data science in action, second edition*. Springer.

Vanderfeesten, I., & Grefen, P. (2015). Advanced dynamic role resolution in business processes. *CAiSE Workshops*, 87–93.

Zhao, W., Liu, H., Dai, W., & Ma, J. (2016). An entropy-based clustering ensemble method to support resource allocation in business process management. *Knowl. Inf. Syst.*, *48*(2).

Ziebart, B. D., Maas, A. L., Bagnell, J. A., & Dey, A. K. (2008). Maximum entropy inverse reinforcement learning. *AAAI*, 1433–1438.